Bloque 3: Integración y despliegue de la red neuronal en el RoboMaster

1 Introducción

En este último bloque llevaremos a cabo la **integración en hardware** del detector entrenado en el Bloque 2. En un proyecto real, integrar implica seleccionar y montar el hardware, adaptar la red a sus limitaciones, optimizar la inferencia en tiempo real, evaluar en entorno controlado y, por último, probar en el mundo real. Aquí nos quedaremos en la integración y la evaluación en un entorno controlado.

Para ello usaremos la **SDK de Python** del RoboMaster, que permite controlar el robot desde el ordenador. Con ella diseñaremos un circuito en el que nuestro robot deberá recorrer un trayecto, buscar RoboMasters adversarios, detectarlos con nuestra red y dispararles para dejarlos fuera de combate. Esto supone decidir cómo se mueve el robot, cómo gira su torreta, cuándo ejecuta la detección y cómo dispara a lo que encuentra. Además, cada grupo podrá añadir funcionalidades propias (luces, sonidos, recorridos más elaborados, etc.) para enriquecer la demo final.

2 VSCode

Como no es posible conectarse a los RoboMasters desde JupyterHub, esta parte la desarrollaremos en VSCode. Abra la aplicación y cargue el proyecto de la carpeta */home/PIDS/robomaster* (figura 1a). Encontrará *connect.py* para conectar el robot, *demo.py* para programar la demo, y varios scripts de ejemplo con los que aprender a manejar cada módulo.

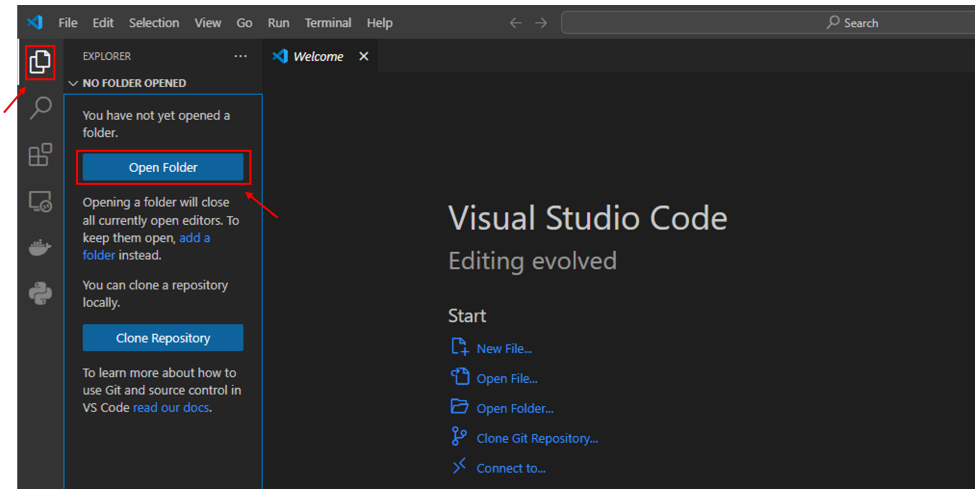
Antes de ejecutar nada, asegúrese de que el intérprete seleccionado es el entorno virtual preparado para la práctica (Python 3.8). Aparece en la esquina inferior derecha (figura 1b). Si no es el correcto, haga click sobre él o pulse Ctrl+Shift+P y escriba "Select Interpreter". La ruta es */home/PIDS/robomaster/.pixi/venv/bin/python*.

Para ejecutar un script, ábralo y pulse Ctrl+F5 seleccionando "Python Debugger", o despliegue el menú junto al botón "Play" de la esquina superior derecha y elija "Run Python File" (la primera vez). A partir de ahí bastará con pulsar el botón "Play" para volver a lanzar el mismo script. Para depurar código, haga click a la izquierda del número de línea donde quiera detenerse; aparecerá un círculo rojo (un *breakpoint*). Al lanzar el script con "Python Debugger", la ejecución se parará ahí y podrá inspeccionar el valor de las variables y avanzar paso a paso con los botones de la barra de depuración. Para detener una ejecución, pulse Shift+F5 o Ctrl+C.

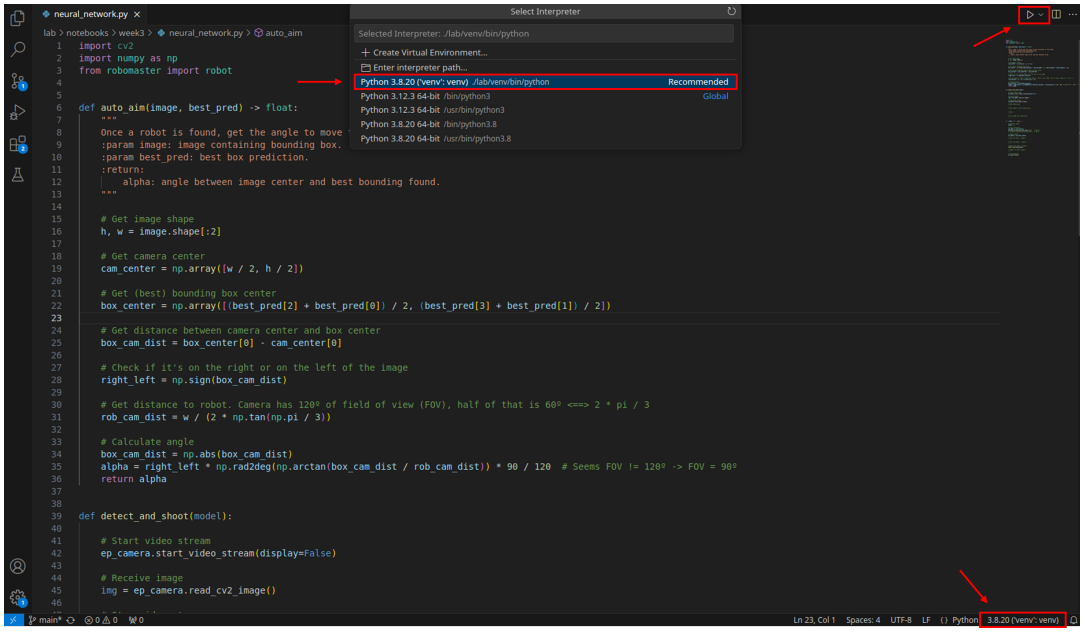
3 Uso del RoboMaster

Los RoboMasters admiten dos modos de conexión: "Wi-Fi" y "networking". El modo Wi-Fi es el que usamos en el Bloque 1: el robot emite su propia red. Como los ordenadores del laboratorio no tienen adaptador Wi-Fi, esta vez usaremos el modo networking, que consiste en mostrar al robot un código QR con los datos de la red del laboratorio para que se conecte a ella.

Abra *connect.py*, escriba la SSID y la contraseña de la red del laboratorio y añada el número de serie de su RoboMaster (recuadro superior de la figura 2). El número de serie no hace falta para conectarse, pero es la única forma de que su ordenador distinga su robot del de otro grupo conectado



(a) Directorio de trabajo



(b) Entorno virtual

Figura 1: Configuración de VSCode.

a la misma red. Sin él podría lanzar su script sobre el robot ajeno y provocar un accidente. El script lo guardará en *serial_number.txt*.

Antes de lanzarlo, prepare el robot (figura 2): deslice la pestaña de conexión hasta el icono del router, que es el modo networking, y pulse el botón rojo que hay junto a ella para dejar al robot a la espera del código QR.

Lance entonces el script, agarre el robot y apunte con cuidado su cabeza hacia el QR emergente (si no aparece, abra el archivo *qrcode.png*). El robot avisará cuando se haya conectado.

3.1 Módulos y scripts de ejemplo

El proyecto incluye uno o varios scripts de ejemplo por cada módulo del robot. Ejecútelos todos y analícelos con calma: son las piezas con las que construirá la demo, y en la demo final deberá usarlos todos. Una vez los domine, podrá completar *demo.py* y *demo_utils.py*, los scripts sobre los que programará el circuito final.



Figura 2: Activar el modo "networking" para conectar el RoboMaster a la red Wi-Fi del laboratorio.

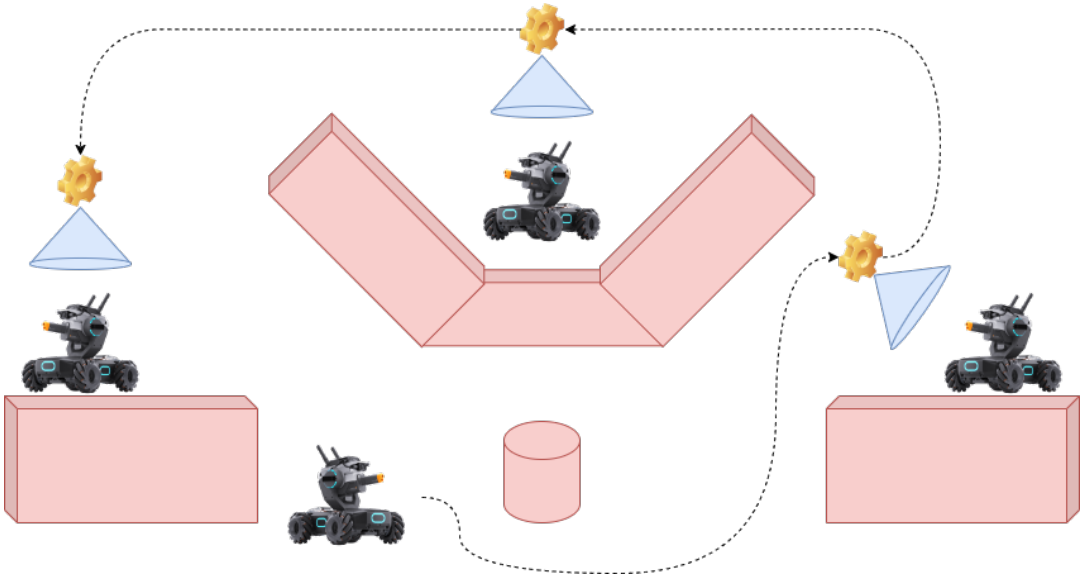


Figura 3: Circuito de ejemplo.

4 El circuito

La evaluación final se hará sobre un circuito que pone a prueba todo lo desarrollado durante el proyecto. Constará de una serie de obstáculos que cada grupo deberá sortear (figura 3) y de puntos estratégicos desde los que buscar robots adversarios con la red del Bloque 2. Los objetivos detectados serán disparados con el blaster hasta inutilizarlos.

La estrategia de recorrido y de detección es la clave de esta semana. Detectar es costoso computacionalmente, así que no se trata de mirar todo el rato, sino de mirar donde y cuando conviene. Una estrategia habitual es la siguiente:

1. **Desplazarse a un punto estratégico**, elegido por su visibilidad y por las posibles posiciones de los adversarios.

2. **Detenerse y capturar una imagen** con la cámara.

3. **Detectar** sobre esa imagen. Si no aparece ningún adversario, girar la torreta un ángulo fijo (por ejemplo, 90 grados), capturar de nuevo y repetir hasta completar un barrido de 360 grados (sin olvidarse después de resetear la posición de la cabeza).

4. **Apuntar y disparar**: quedarse con las detecciones cuya confianza supere un umbral, usar la función de autoapuntado de *demo.py* para calcular el ángulo y disparar con el blaster.

Es solo un ejemplo. Cada grupo diseñará su propia estrategia, adaptándola a su circuito y a los recursos disponibles.

5 Objetivos

Hay libertad para diseñar el circuito, pero estos puntos son obligatorios:

- **Recorrido:** dado un punto inicial y una serie de puntos de parada, diseñe un trayecto que pase por todos ellos **sin colisionar** con ningún obstáculo.
- **Objetivos:** inutilice a todos los RoboMasters del circuito (hacen falta 12 disparos por robot). Se entiende que el detector puede fallar en algún punto, sobre todo por la escasez de muestras de entrenamiento. No olvide guardarse las detecciones realizadas de cara al entregable.
- **Estrategia de detección:** piense bien qué hacer en cada parada. ¿Disparar solo a la detección de mayor confianza o a todas las que superen el umbral? ¿Seguir girando la torreta hasta detectar algo o hasta completar los 360 grados?
- **Evidencias:** cada imagen sobre la que se aplique el detector debe guardarse en una carpeta dedicada, con las cajas predichas y su confianza dibujadas encima. Las necesitará para la memoria.
- **Módulos:** use obligatoriamente todos los módulos del robot: chasis, torreta, cámara, blaster, LEDs y audio.
- **Originalidad:** se valoran los circuitos elaborados, los bailes, los obstáculos extra y, en general, cualquier añadido que mejore la demo.

NOTA: trate a los robots con sumo cuidado. Está **PROHIBIDO** aumentar su velocidad. No confunda los ejes "x" e "y" (deje espacio en todas direcciones al hacer pruebas) y, cuando lance un script que mueva el robot, que un compañero lo siga de cerca para evitar colisiones; detenga el script de inmediato (Ctrl+C) ante cualquier riesgo. La torreta es delicada: no agarre nunca el robot por la cabeza, siempre por la base. Y no apunte a los ojos de nadie con el blaster.

5.1 Entregable

La memoria es **incremental**: esta es la última entrega y cierra el documento. Siga utilizando la plantilla *PIDS-template.docx*. Para esta tercera entrega:

- **Contenido:** los Bloques 1 y 2 corregidos, el Bloque 3 completo y las conclusiones generales del proyecto. Rellene también la tabla "Respuesta a la revisión anterior": su ausencia penaliza la nota.

- **Extensión máxima:** 30 páginas de cuerpo, contando los Bloques 1 y 2. No cuentan la portada, el índice, la respuesta a la revisión anterior, el reparto del trabajo, las referencias ni los anexos.
- **Fichero:** un único PDF llamado **PIDS_grupoX_entregaN.pdf**, donde X es el número de grupo y N el de la entrega. Por ejemplo, el grupo 3 entregaría *PIDS_grupo3_entrega3.pdf*.

Se valorará el análisis honesto de la demo: una demostración imperfecta bien explicada puntúa más que una perfecta sin analizar correctamente.